Parte 1

Verificación visual.

In [242…
```
%matplotlib inline
pts = [(0, 1), (1, 5), (2, 3)]
pts2 = [(0, -5), (1, -4), (2, 3)]
pts3 = [(0, -1), (1, 1), (2, 5), (3, 2)]
```

In [243…
```
def Spline(x, xi, *, a, b, c, d):
    return a + b * (x - xi) + c * (x - xi) ** 2 + d * (x - xi) ** 3
```

In [244…
```
import matplotlib.pyplot as plt
import numpy as np


def dibujar_spline(pts, s):
    xs, ys = zip(*pts)
    for i, x_i in enumerate(xs[:-1]):
        _x = np.linspace(x_i, xs[i + 1], 20)
        _y = Spline(_x, x_i, **s[i])
        plt.plot(_x, _y)

    plt.scatter(xs, ys)
    plt.xlabel("x")
    plt.ylabel("y")
    plt.title("Interpolación con splines cúbicos")
    plt.show()
```

In [245…
```
import matplotlib.pyplot as plt
import numpy as np


def dibujar_spline(pts2, s1):
    xs, ys = zip(*pts2)
    for i, x_i in enumerate(xs[:-1]):
        _x = np.linspace(x_i, xs[i + 1], 20)
        _y = Spline(_x, x_i, **s1[i])
        plt.plot(_x, _y)

    plt.scatter(xs, ys)
    plt.xlabel("x")
    plt.ylabel("y")
    plt.title("Interpolación con splines cúbicos")
    plt.show()
```

In [246…
```
import matplotlib.pyplot as plt
import numpy as np


def dibujar_spline(pts3, s2):
    xs, ys = zip(*pts3)
    for i, x_i in enumerate(xs[:-1]):
        _x = np.linspace(x_i, xs[i + 1], 20)
        _y = Spline(_x, x_i, **s2[i])
        plt.plot(_x, _y)

    plt.scatter(xs, ys)
    plt.xlabel("x")
    plt.ylabel("y")
```

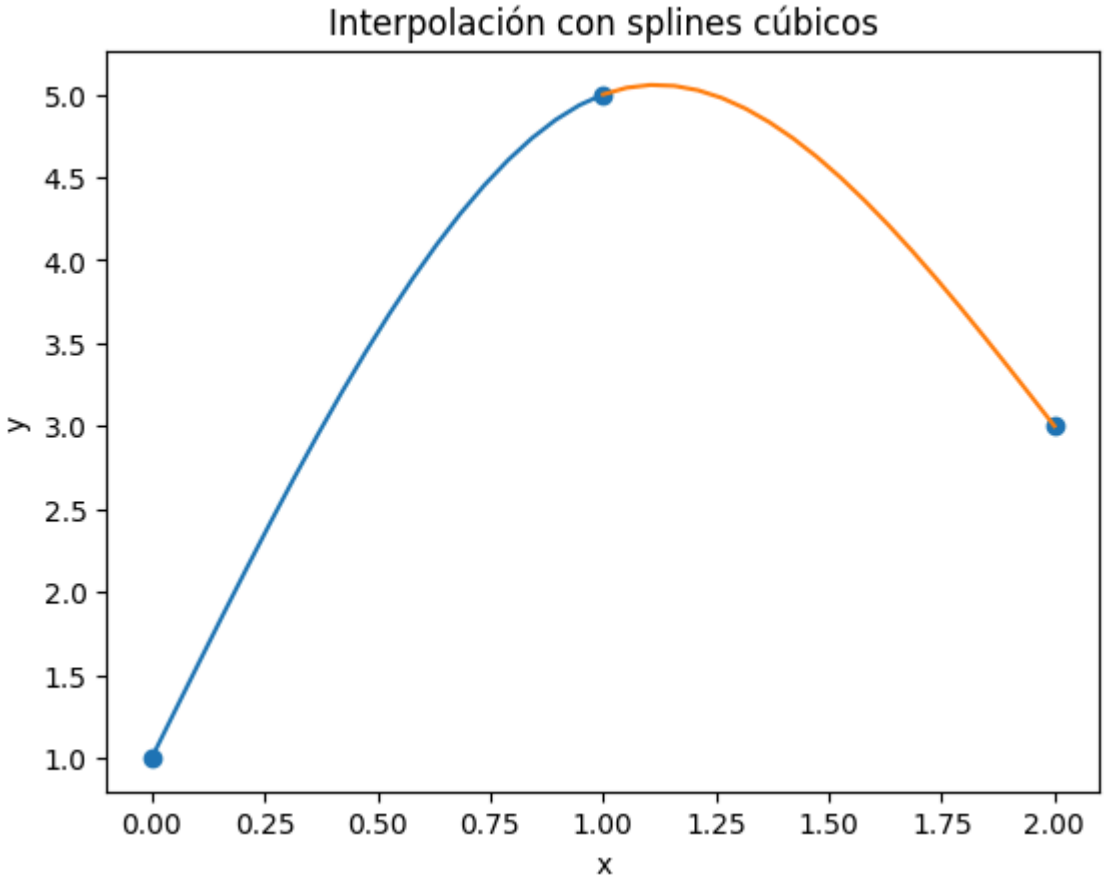
```
    plt.title("Interpolación con splines cúbicos")
    plt.show()
```

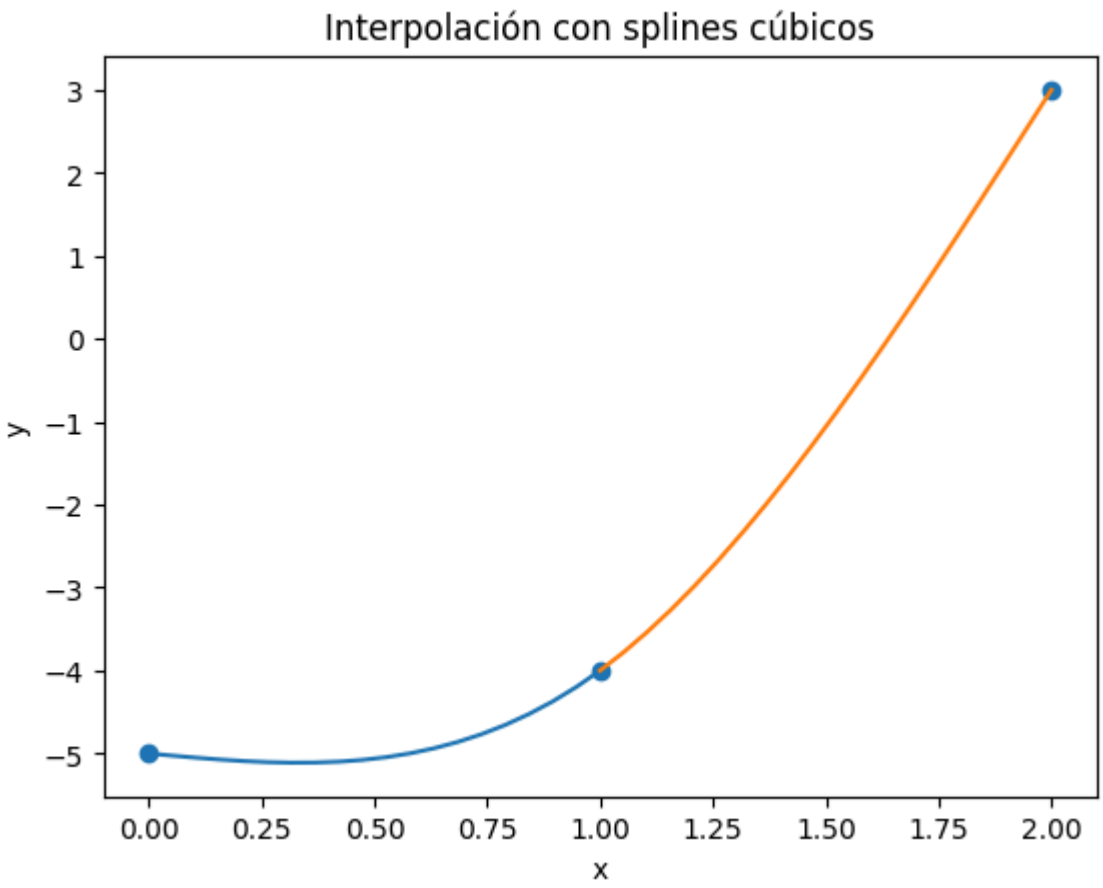
In [247…

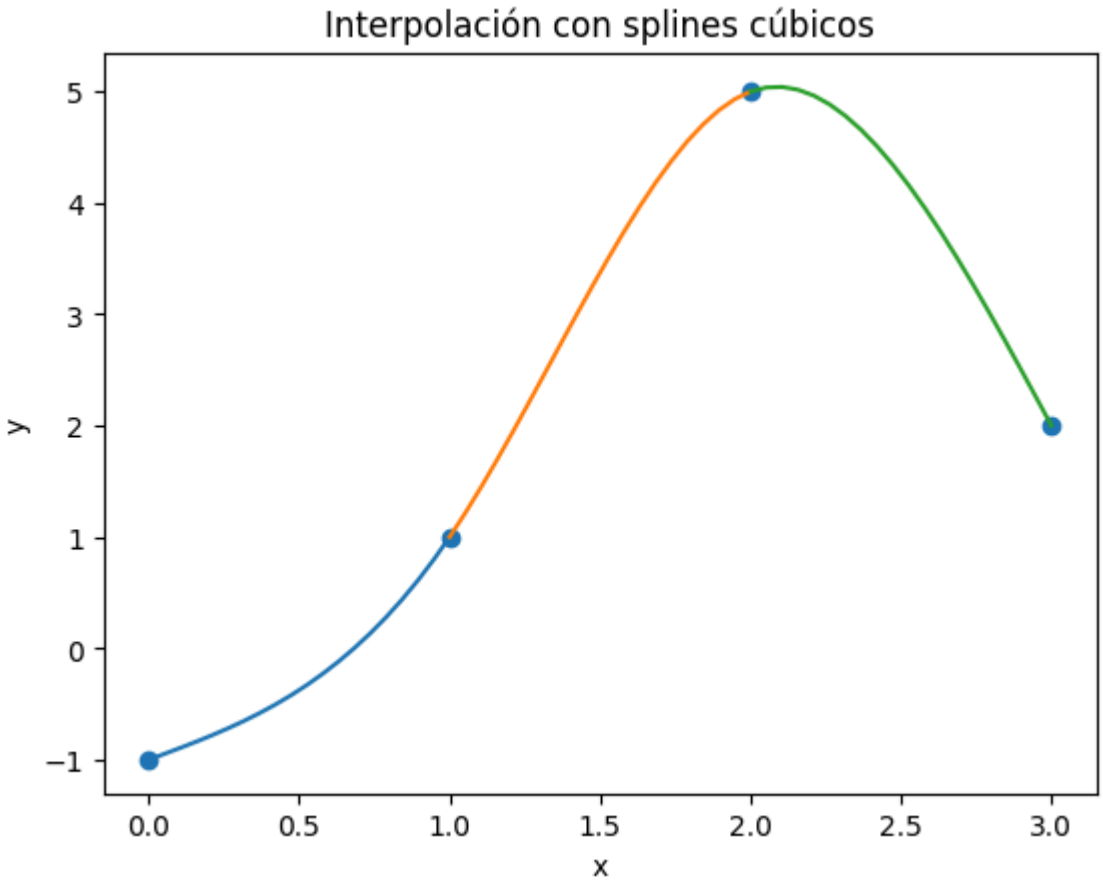
```
s = [
    {"a": 1, "b": 5.5, "c": 0, "d": -1.5},
    {"a": 5, "b": 1, "c": -4.5, "d": 1.5},
]

s1 = [
    {"a": -5, "b": -0.5, "c": 0, "d": 1.5},
    {"a": -4, "b": 4, "c": 4.5, "d": -1.5},
]

s2 = [
    {"a": -1, "b": 1, "c": 0, "d": 1},
    {"a": 1, "b": 4, "c": 3, "d": -3},
    {"a": 5, "b": 1, "c": -6, "d": 2},
]
dibujar_spline(pts, s)
dibujar_spline(pts2, s1)
dibujar_spline(pts3, s2)
```







Parte 2

Completar código para definir la función Spline.

In [248…

```
import sympy as sym
from IPython.display import display
```

```python
# ########################################################################
def cubic_spline(xs: list[float], ys: list[float]) -> list[sym.Symbol]:
    """
    Cubic spline interpolation ``S``. Every two points are interpolated by a cubic polynomial
    ``S_j`` of the form ``S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3.``

    xs must be different  but not necessarily ordered nor equally spaced.

    ## Parameters
    - xs, ys: points to be interpolated

    ## Return
    - List of symbolic expressions for the cubic spline interpolation.
    """

    points = sorted(zip(xs, ys), key=lambda x: x[0])  # sort points by x

    xs = [x for x, _ in points]
    ys = [y for _, y in points]

    n = len(points) - 1  # number of splines

    h = [xs[i + 1] - xs[i] for i in range(n)]  # distances between  contiguous xs

    alpha = [0] * (n + 1) ## Es dado para completar por los numero de puntos, basicamente el
    for i in range(1, n):
        alpha[i] = 3 / h[i] * (ys[i + 1] - ys[i]) - 3 / h[i - 1] * (ys[i] - ys[i - 1])

    l = [1]
    u = [0]
    z = [0]

    for i in range(1, n):
        l += [2 * (xs[i + 1] - xs[i - 1]) - h[i - 1] * u[i - 1]]
        u += [h[i] / l[i]]
        z += [(alpha[i] - h[i - 1] * z[i - 1]) / l[i]] # z es el resultado de la sustitucion

    l.append(1)
    z.append(0)
    c = [0] * (n + 1)

    x = sym.Symbol("x")
    splines = []
    for j in range(n - 1, -1, -1):
        c[j] = z[j] - u[j] * c[j + 1]
        b = (ys[j + 1] - ys[j]) / h[j] - h[j] * (c[j + 1] + 2 * c[j]) / 3
        d = (c[j + 1] - c[j]) / (3 * h[j])
        a = ys[j]
        print(j, a, b, c[j], d)
        S  = a + b * (x - xs[j]) + c[j] * (x - xs[j]) ** 2 + d * (x - xs[j]) ** 3

        splines.append(S)
    splines.reverse()
    return splines
```

In [249…

```python
xs = [0, 1, 2]
ys = [-5, -4, 3]

splines = cubic_spline(xs=xs, ys=ys)
_ = [display(s) for s in splines]
print("______")
_ = [display(s.expand()) for s in splines]
```

```
1 -4 4.0 4.5 -1.5
0 -5 -0.5 0.0 1.5
```

$1.5x^3 - 0.5x - 5$

$4.0x - 1.5(x-1)^3 + 4.5(x-1)^2 - 8.0$

$1.5x^3 - 0.5x - 5$

$-1.5x^3 + 9.0x^2 - 9.5x - 2.0$

In [250…

```
xs = [0, 1, 2]
ys = [-5, -4, 3]

splines = cubic_spline(xs=xs, ys=ys)
_ = [display(s) for s in splines]
print("______")
_ = [display(s.expand()) for s in splines]
```

```
1 -4 4.0 4.5 -1.5
0 -5 -0.5 0.0 1.5
```

$1.5x^3 - 0.5x - 5$

$4.0x - 1.5(x-1)^3 + 4.5(x-1)^2 - 8.0$

$1.5x^3 - 0.5x - 5$

$-1.5x^3 + 9.0x^2 - 9.5x - 2.0$

In [251…

```
xs = [0, 1, 2]
ys = [1,5,3]
xs2= [0, 1, 2]
ys2= [-5,4,3]
xs3= [0, 1, 2, 3]
ys3= [-1,1,5,2]
#realizar la impresion un rango-primero separar ejecucion en un if de ejecuicion por rango

splines1 = cubic_spline(xs=xs, ys=ys)
splines2 = cubic_spline(xs=xs2, ys=ys2)
splines3 = cubic_spline(xs=xs3, ys=ys3)
_ = [display(s) for s in splines1]
print("______")
_ = [display(s.expand()) for s in splines1]
```

```
1 5 1.0 -4.5 1.5
0 1 5.5 0.0 -1.5
1 4 4.0 -7.5 2.5
0 -5 11.5 0.0 -2.5
2 5 1.0 -6.0 2.0
1 1 4.0 3.0 -3.0
0 -1 1.0 0.0 1.0
```

$-1.5x^3 + 5.5x + 1$

$1.0x + 1.5(x-1)^3 - 4.5(x-1)^2 + 4.0$

$-1.5x^3 + 5.5x + 1$

$1.5x^3 - 9.0x^2 + 14.5x - 2.0$

In [252…

```
splines[1]
```

Out[252…

$4.0x - 1.5(x-1)^3 + 4.5(x-1)^2 - 8.0$

In [253…
```
print("______")
splines[0]
```

```
______
```

Out[253… $1.5x^3 - 0.5x - 5$

In [254…
```
def evaluar_spline1(splines, xv):

    # usar el símbolo 'x' ya definido en el notebook para la sustitución
    try:
        sym_x = x
    except NameError:
        sym_x = sym.Symbol("x")

    xv = float(xv)
    # asumir que 'xs' está definido en el notebook (lista de nodos ordenada)
    for j in range(len(xs) - 1):
        if xs[j] <= xv <= xs[j + 1]:
            return float(splines[j].subs(sym_x, xv))
    raise ValueError(f"x={xv} fuera del rango [{xs[0]}, {xs[-1]}]")
```

In [255…
```
def evaluar_spline2(splines2, xv):
    try:
        sym_x = x
    except NameError:
        sym_x = sym.Symbol("x")

    xv = float(xv)
    for j in range(len(xs2) - 1):
        if xs2[j] <= xv <= xs2[j + 1]:
            return float(splines2[j].subs(sym_x, xv))
    raise ValueError(f"x={xv} fuera del rango [{xs2[0]}, {xs2[-1]}]")

def evaluar_spline3(splines3, xv):
    try:
        sym_x = x
    except NameError:
        sym_x = sym.Symbol("x")

    xv = float(xv)
    for j in range(len(xs3) - 1):
        if xs3[j] <= xv <= xs3[j + 1]:
            return float(splines3[j].subs(sym_x, xv))
    raise ValueError(f"x={xv} fuera del rango [{xs3[0]}, {xs3[-1]}]")
```

In [256…
```
x = sym.Symbol("x")
splines1[1].subs(x, 0)
```

Out[256… -2.0

In [257…
```
splines1[0].subs(x, 5)
```

Out[257… -159.0

In [258…
```
splines2[0].subs(x, 0)
```

Out[258… -5

In [259…
```
splines2[1].subs(x, 2)
```

Out[259… 3.0

In [260…
```
splines3[0].subs(x, 1)
```

Out[260…
$$1.0$$

In [261…
```
splines3[1].subs(x, 1.5)
```

Out[261…
$$3.375$$

In [262…
```
splines3[2].subs(x, 2.5)
```

Out[262…
$$4.25$$